

Lekki system gestów dłoni do przewijania slajdów

Podsumowanie wykonawcze

Dla programu działającego w tle podczas prezentacji, który ma rozpoznawać gest dłoni przed ekranem i wysyłać `Space`, najlepszy bilans między **małym zużyciem CPU/GPU**, **brakiem dodatkowego GPU**, **przenośnością** i **prostotą wdrożenia** daje dziś wariant **kamera RGB + MediaPipe Hand Landmarker** albo jeszcze lżejszy, ale mniej odporny, **heurystyczny OpenCV**. MediaPipe jest projektowane do działania on-device i ma oficjalne taski dla Pythona; w trybach `VIDEO` i `LIVE_STREAM` używa śledzenia między klatkami, aby nie uruchamiać pełnej detekcji na każdej klatce, a oficjalny benchmark Gesture Recognizer pokazuje średnią latencję CPU rzędu 16.76 ms na Pixel 6, co jest dobrą wskazówką, że na typowym laptopie bez dGPU rozwiązanie jest praktyczne przy niskiej rozdzielczości wejścia. ¹

Jeżeli dopuszczasz **dodatkowy sensor**, to jakościowo najlepszą ścieżką dla prezentera jest zwykle **Ultraleap Leap Motion Controller 2**: to sprzęt wyspecjalizowany w śledzeniu dłoni, z minimalnymi wymaganiami hosta już od Windows 10, Core i3 5. generacji, 2 GB RAM i USB 2.0. Dla zastosowań „nad ekranem” Ultraleap ma nawet gotową architekturę produktu TouchFree, który działa „niewidocznie” nad istniejącym interfejsem i komunikuje się z wejściem systemowym Windows. Alternatywą z depth jest **Intel RealSense**: ma cross-platform SDK, a obliczenia depth są wykonywane na kamerze przez układ D4, co zmniejsza obciążenie hosta. ²

Radar typu Soli, ultradźwięki i wearable IMU są ciekawe, ale nie są moją pierwszą rekomendacją dla prostego sterownika prezentacji na laptopie. Radar/Soli ma bardzo dobre właściwości prywatnościowe i energetyczne, ale obecnie jest raczej kierunkiem produktowo-badawczym niż wygodną ścieżką desktopową; oficjalne zastosowania konsumenckie to przede wszystkim Pixel 4 i Nest Hub. Ultradźwięki i IMU potrafią być bardzo wydajne obliczeniowo, ale zwykle wymagają własnego hardware’u lub elementu noszonego przez użytkownika, co podnosi tarcie UX. ³

Wniosek praktyczny: jeśli chcesz zacząć szybko i tanio, buduj **MediaPipe + pynput**. Jeśli zależy Ci na najwyższej niezawodności w prezentacjach live i akceptujesz koszt sprzętu, idź w **Ultraleap**. **RealSense** wybieraj wtedy, gdy scena jest trudna, tło ruchliwe albo nie chcesz polegać wyłącznie na RGB. ⁴

Które gesty i które czujniki mają sens

Dla sterowania slajdami najlepiej sprawdza się **gest dynamiczny**, a nie statyczna poza. Z tego powodu **swipe** jest zwykle lepszym domyślnym wyborem niż samo **open/close palm**. Dynamiczny ruch daje silniejszy sygnał czasowy, łatwiej go odróżnić od zwykłego trzymania ręki w polu widzenia, a dodatkowo jest naturalny dla większości modalności: RGB, depth, Leap/IR i radar. W literaturze radarowej zestaw gestów często obejmuje właśnie **cztery kierunki swipe** oraz **push**; w MediaPipe Gesture Recognizer gotowe klasy obejmują za to `Open_Palm` i `Closed_Fist`, ale nie ma gotowego `Swipe`, więc dla swipe i tak zwykle trzeba dodać logikę czasową nad landmarkami. ⁵

Moja rekomendacja UX dla prezentacji jest następująca: używaj **jednego zamierzonego gestu głównego** do przejścia dalej, najlepiej **poziomego swipe** wykrywanego w krótkim oknie czasu, oraz

ewentualnie **statycznego gestu uzbrajającego** jako warunku bezpieczeństwa, np. „otwarta dłoń przez 150–250 ms, a potem swipe”. Jeżeli chcesz maksymalnie zredukować false positive, **wave** traktuj raczej jako gest potwierdzający niż podstawowy; falowanie jest bardziej podobne do spontanicznej gestykulacji prezentera. To jest synteza inżynierska oparta na tym, że MediaPipe ma gotowe klasy dla gestów statycznych, a platformy radarowe i depth dobrze chwytają ruch kierunkowy. ⁶

Pod względem czujników różnica jest zasadnicza. **RGB webcam** jest najtańszy i najłatwiejszy, ale najmniej odporny na oświetlenie i tło. **OpenCV optical flow** zakłada stałość jasności piksela między kolejnymi klatkami oraz podobny ruch sąsiednich pikseli, a **background subtraction** zakłada w praktyce kamerę statyczną. **Depth cameras** są bardziej odporne na złożone tła i separują rękę w osi Z; **RealSense** deklaruje pracę indoor/outdoor i szeroki zakres warunków oświetleniowych. **Ultraleap** używa kamer stereo IR i diod IR, więc jest bardzo dobre w krótkim dystansie nad ekranem, ale dokumentacja wprost ostrzega przed jasnym słońcem, odbiciami IR i refleksami od ekranów. **Radar** ma przewagę prywatnościową, bo nie generuje rozróżnialnych obrazów osoby. **IMU** i **ultradźwięki** są obliczeniowo lekkie, ale albo wymagają wearables, albo customowej elektroniki. ⁷

Ważny niuans produktowy: jeśli Twoim celem jest **daemon działający w tle**, a nie eksperyment CV, to wzorcem architektonicznym jest raczej **lekki proces wejściowy + prosty stan gestu + keyboard injection**, a nie ciężki klasyfikator sekwencyjny. To właśnie dlatego OpenCV i MediaPipe są tu tak sensowne, a niekoniecznie PyTorch/TensorFlow z własnym modelem. MediaPipe i ONNX Runtime/TFLite mają sens dopiero wtedy, gdy naprawdę potrzebujesz trenować własne gesty lub personalizować model. ⁸

Porównanie podejść i sprzętu

sensing	libs/frameworks	accuracy	latency	CPU load estimate	pros/cons	cost	
RGB webcam heurystyczny	OpenCV, <code>pynput</code> / <code>pyautogui</code>	niska–średnia w kontrolowanych warunkach	bardzo niska	bardzo niski	+ najlżejszy, zero ML, działa na zwykłym CPU; – wrażliwy na światło, ruch kamery i tło	bardzo niski, zwykle brak dodatkowego kosztu	b c
RGB webcam + landmarks	MediaPipe Hand Landmarker, OpenCV, <code>pynput</code>	średnia–wysoka	niska	niski–średni	+ najlepszy stosunek jakości do ceny, landmarki i handedness; – nadal RGB i zależność od jakości kadru	bardzo niski	b c
RGB webcam + gotowe gesty statyczne	MediaPipe Gesture Recognizer	wysoka dla <code>Open_Palm</code> / <code>Closed_Fist</code> , średnia dla custom swipe	niska	niski–średni	+ gotowe gesty statyczne; – swipe wymaga własnej logiki lub custom modelu	bardzo niski	b c

sensing	libs/frameworks	accuracy	latency	CPU load estimate	pros/cons	cost	
Leap Motion Controller 2	Ultraleap runtime, LeapC, Python bindings	wysoka	bardzo niska	niski na hoście	+ sensor wyspecjalizowany, dobry nad ekranem, małe wymagania hosta; – koszt, dodatkowy hardware, Python bindings mniej gładkie niż SDK C#/C++	średni- wysoki	
Depth camera	RealSense SDK 2.0, pyrealsense2, OpenCV	wysoka dla segmentacji ręki	niska	niski- średni	+ separacja dłoni po głębi, lepsza odporność na tło; – droższe i większe niż potrzeba do jednego gestu	wysoki	
Radar 60 GHz / Soli- class	custom SDK / research stack	wysoka w zadaniach laboratoryjnych	bardzo niska	bardzo niski	+ prywatność, low power, dobra praca bez obrazu; – bardzo słaba dostępność desktopowa	wysoki / brak commodity	
Ultrasonic	custom DSP / research	średnia-wysoka w prototypach	niska	niski	+ dobra prywatność, niskie wymagania obliczeniowe; – mała dostępność, zwykle custom hardware	zależny od prototypu	
Wearable IMU	BLE + Python, własny firmware/ app	wysoka dla gestów zdefiniowanych pod dany wearable	bardzo niska	bardzo niski	+ prywatność, odporność na światło, małe obciążenie hosta; – trzeba coś nosić, większe tarcie UX	niski-średni dla modułów, wyższy dla gotowych wearables	

Ta tabela jest **syntetyczną oceną inżynierską**. Wiersze dotyczące OpenCV wynikają bezpośrednio z założeń optical flow i background subtraction oraz z cross-platformowości OpenCV. Wiersze MediaPipe opierają się na oficjalnych taskach, benchmarkach i logice śledzenia między klatkami. Wiersze Ultraleap i RealSense wynikają z dokumentacji sprzętu i SDK. Wiersze radar/ultradźwięki/IMU opierają się na publikacjach, w których osiągnięto odpowiednio m.in. 98.4% F1 dla lekkiego radaru FMCW na edge,

89.38%/96.34% dla HUG na ultradźwiękach oraz 92% i 63 ms dla rękawicy IMU; tych wyników nie należy jednak bezpośrednio przenosić na spontaniczne gesty prezentera bez własnej walidacji. ⁹

Jeżeli mam to uprościć do jednej decyzji: **MediaPipe + webcam** wybieraj, gdy chcesz dojść do działającego MVP w kilka-kilkanaście godzin i nie kupować sprzętu. **Ultraleap** wybieraj, gdy niezawodność gestu jest ważniejsza niż koszt i chcesz najbliżej podejść do „produktowego” feelingu nad ekranem. **RealSense** wybieraj, gdy zależy Ci na odporności na tło i osi Z albo już masz taki sensor w organizacji. **Soli, ultradźwięki i IMU** trzymaj jako ścieżki specjalistyczne. ¹⁰

Implementacje referencyjne w Pythonie

Wspólny sposób wysyłania klawisza

Najprostszy i najbardziej praktyczny sposób wysłania `Space` z Pythona to `pynput.keyboard.Controller`; alternatywnie możesz użyć `pyautogui`, które również jest cross-platform. `pynput` ma backendi `darwin`, `win32`, `xorg` i opcjonalny `uinput` dla Linuksa; `pyautogui` działa na Windows, macOS i Linux. ¹¹

```
from pynput.keyboard import Controller, Key

keyboard = Controller()
keyboard.press(Key.space)
keyboard.release(Key.space)

# alternatywa:
# import pyautogui
# pyautogui.press("space")
```

Heurystyka OpenCV z webcam i bardzo niskim kosztem CPU

To jest najlżejsza wersja: działa bez ML, używa **background subtraction** do wycięcia ruchomego obiektu i **optycznego przepływu** do potwierdzenia kierunku ruchu. Jest szybka, ale zakłada względnie statyczną kamerę i dość stabilne światło. OpenCV udostępnia dokładnie te prymitywy: `VideoCapture`, `BackgroundSubtractorMOG2/KNN` oraz optical flow (`Lucas-Kanade`, `Farneback`). ¹²

Zależności

```
pip install opencv-python numpy pynput
```

Kod

```
import time
from collections import deque

import cv2
import numpy as np
from pynput.keyboard import Controller, Key
```

```

keyboard = Controller()

# --- parametry ---
CAM_INDEX = 0
FRAME_W, FRAME_H = 640, 360
COOLDOWN_S = 0.9
MIN_CONTOUR_AREA = 2500
MAX_HISTORY = 8

# ROI: środkowa część obrazu; dobrą praktyką jest zawężenie pola obserwacji
ROI_X0, ROI_Y0 = 0.15, 0.20
ROI_X1, ROI_Y1 = 0.85, 0.80

cap = cv2.VideoCapture(CAM_INDEX)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, FRAME_W)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_H)

backsub = cv2.createBackgroundSubtractorMOG2(
    history=200, varThreshold=32, detectShadows=False
)

centers = deque(maxlen=MAX_HISTORY)
prev_gray = None
last_trigger = 0.0

def press_space():
    keyboard.press(Key.space)
    keyboard.release(Key.space)

while True:
    ok, frame = cap.read()
    if not ok:
        break

    # mirror jak w kamerce do prezentacji
    frame = cv2.flip(frame, 1)
    h, w = frame.shape[:2]

    x0, y0 = int(w * ROI_X0), int(h * ROI_Y0)
    x1, y1 = int(w * ROI_X1), int(h * ROI_Y1)
    roi = frame[y0:y1, x0:x1]

    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)

    # foreground mask z background subtraction
    fg = backsub.apply(gray)
    fg = cv2.GaussianBlur(fg, (5, 5), 0)
    _, fg = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY)
    kernel = np.ones((5, 5), np.uint8)
    fg = cv2.morphologyEx(fg, cv2.MORPH_OPEN, kernel)

```

```

fg = cv2.morphologyEx(fg, cv2.MORPH_CLOSE, kernel)

mean_flow_x = 0.0
if prev_gray is not None:
    flow = cv2.calcOpticalFlowFarneback(
        prev_gray, gray, None,
        pyr_scale=0.5, levels=3, winsize=15,
        iterations=3, poly_n=5, poly_sigma=1.2, flags=0
    )
    active = fg > 0
    if np.count_nonzero(active) > 200:
        mean_flow_x = float(np.mean(flow[..., 0][active]))
prev_gray = gray

contours, _ = cv2.findContours(fg, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

if contours:
    c = max(contours, key=cv2.contourArea)
    area = cv2.contourArea(c)

    if area > MIN_CONTOUR_AREA:
        M = cv2.moments(c)
        if M["m00"] > 0:
            cx = int(M["m10"] / M["m00"])
            cy = int(M["m01"] / M["m00"])
            centers.append((time.monotonic(), cx, cy))

            # debug overlay
            cv2.drawContours(roi, [c], -1, (0, 255, 0), 2)
            cv2.circle(roi, (cx, cy), 5, (0, 0, 255), -1)

now = time.monotonic()
if len(centers) >= 5 and (now - last_trigger) > COOLDOWN_S:
    t0, cx0, cy0 = centers[0]
    t1, cx1, cy1 = centers[-1]
    dt = max(t1 - t0, 1e-3)
    dx = cx1 - cx0
    dy = cy1 - cy0
    vx = dx / dt

    horizontal = abs(dx) > 0.28 * (x1 - x0) and abs(dx) > 2.0 * abs(dy)
    flow_confirms = (abs(mean_flow_x) < 0.2) or (np.sign(mean_flow_x) ==
np.sign(dx))
    fast_enough = abs(vx) > 150

    if horizontal and flow_confirms and fast_enough:
        press_space()
        last_trigger = now
        centers.clear()

```

```

# debug:
cv2.rectangle(frame, (x0, y0), (x1, y1), (255, 255, 0), 2)
cv2.putText(frame, f"flow_x={mean_flow_x:.2f}", (10, 25),
             cv2.FONT_HERSHEY_SIMPLEX, 0.7, (50, 220, 50), 2)
cv2.putText(frame, "q = quit", (10, 50),
             cv2.FONT_HERSHEY_SIMPLEX, 0.7, (50, 220, 50), 2)
cv2.imshow("gesture-debug", frame)

if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
cv2.destroyAllWindows()

```

Kiedy ten wariant ma sens

Ten wariant ma sens wtedy, gdy prezentujesz w przewidywalnym środowisku: siedzisz lub stoisz podobnie, tło jest spokojne, a dłoń pojawia się mniej więcej w tym samym miejscu. To najniższy koszt obliczeniowy i najprostsza ścieżka do działającego MVP. Jego słabość wynika wprost z natury optical flow i background subtraction: ruch kamery, zmianę światła, przechodzącą osobę w tle albo animacje na ekranie łatwo pomylić z gestem. ¹³

Jak rozszerzyć ten wariant

Jeśli chcesz wykrywać **wave**, licz zmiany znaku `mean_flow_x` albo `dx` w krótkim oknie czasu i wymagaj 2–3 oscylacji przed wysłaniem klawisza. Jeżeli chcesz wykrywać **open/close**, ten wariant nie jest idealny; dużo lepiej przejść do MediaPipe. To zalecenie wynika z tego, że OpenCV w tym podejściu pracuje na ruchu i foreground mask, a nie na geometrii dłoni. ¹⁴

MediaPipe Hand Landmarker z wykrywaniem lewego i prawego swipe

To jest mój domyślny wariant produkcyjny bez dodatkowego hardware'u. MediaPipe Hand Landmarker dla Pythona zwraca **21 landmarków dłoni**, **handedness** i działa w trzech trybach (`IMAGE`, `VIDEO`, `LIVE_STREAM`). W trybach `VIDEO` i `LIVE_STREAM` używa śledzenia, aby nie uruchamiać detekcji palm na każdej klatce, a w trybie live stream nowe klatki mogą być ignorowane, jeśli task jest zajęty — właśnie po to, by nie pompować latencji. ¹⁵

Zależności

```
pip install mediapipe opencv-python numpy pynput
```

Musisz też pobrać lokalnie model task, np. `hand_landmarker.task`, bo oficjalny Python API wymaga wskazania `model_asset_path`. Jeżeli chcesz rozpoznawać gotowe gesty statyczne (`Open_Palm`, `Closed_Fist`, `Thumb_Up` itd.), użyj zamiast tego `GestureRecognizer` i modelu `gesture_recognizer.task`. ¹⁶

Kod

```

import time
from collections import deque

import cv2
import numpy as np
import mediapipe as mp
from pynput.keyboard import Controller, Key
from mediapipe.tasks import python
from mediapipe.tasks.python import vision

keyboard = Controller()

MODEL_PATH = "hand_landmarker.task" # pobierz model i ustaw ścieżkę
FRAME_W, FRAME_H = 640, 360
COOLDOWN_S = 0.8
FRAME_SKIP = 1 # 1 = przetwarzaj co 2. klatkę

BaseOptions = mp.tasks.BaseOptions
HandLandmarker = vision.HandLandmarker
HandLandmarkerOptions = vision.HandLandmarkerOptions
RunningMode = vision.RunningMode

history = deque(maxlen=8)
last_trigger = 0.0

def press_space():
    keyboard.press(Key.space)
    keyboard.release(Key.space)

def palm_center(landmarks):
    # wrist + MCP palców
    idx = [0, 5, 9, 13, 17]
    xs = [landmarks[i].x for i in idx]
    ys = [landmarks[i].y for i in idx]
    return float(np.mean(xs)), float(np.mean(ys))

cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, FRAME_W)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_H)

options = HandLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=MODEL_PATH),
    running_mode=RunningMode.VIDEO,
    num_hands=1,
    min_hand_detection_confidence=0.6,
    min_hand_presence_confidence=0.6,
    min_tracking_confidence=0.6,
)

with HandLandmarker.create_from_options(options) as landmarker:

```

```

frame_idx = 0

while True:
    ok, frame = cap.read()
    if not ok:
        break

    frame = cv2.flip(frame, 1)
    debug = frame.copy()

    if frame_idx % (FRAME_SKIP + 1) != 0:
        frame_idx += 1
        cv2.imshow("mediapipe-gesture-debug", debug)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
        continue

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
    timestamp_ms = int(time.time() * 1000)

    result = landmarker.detect_for_video(mp_image, timestamp_ms)
    now = time.monotonic()

    if result.hand_landmarks:
        lms = result.hand_landmarks[0]
        cx, cy = palm_center(lms)

        xs = [p.x for p in lms]
        ys = [p.y for p in lms]
        hand_w = max(xs) - min(xs)
        hand_h = max(ys) - min(ys)

        history.append((now, cx, cy, hand_w, hand_h))

    # debug rysowanie landmarków
    for p in lms:
        px = int(p.x * frame.shape[1])
        py = int(p.y * frame.shape[0])
        cv2.circle(debug, (px, py), 2, (0, 255, 0), -1)

    if len(history) >= 6 and (now - last_trigger) > COOLDOWN_S:
        t0, x0, y0, w0, h0 = history[0]
        t1, x1, y1, w1, h1 = history[-1]
        dx = x1 - x0
        dy = y1 - y0
        dt = max(t1 - t0, 1e-3)

        # próg względny do rozmiaru dłoni w kadrze
        scale = max((w0 + w1) / 2, 0.08)

```

```

        is_horizontal = abs(dx) > max(0.18, 1.2 * scale) and abs(dx)
> 2.5 * abs(dy)
        is_fast = abs(dx / dt) > 0.35

        if is_horizontal and is_fast:
            direction = "RIGHT" if dx > 0 else "LEFT"
            print(f"Swipe detected: {direction}")

            # Domyślnie oba kierunki wysyłają Space.
            # Jeśli chcesz "poprzedni slajd", zamień LEFT na
Key.left.

            press_space()

            last_trigger = now
            history.clear()

        cv2.putText(debug, "q = quit", (10, 25),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (50, 220, 50), 2)
        cv2.imshow("mediapipe-gesture-debug", debug)

        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

        frame_idx += 1

    cap.release()
    cv2.destroyAllWindows()

```

Dlaczego to zwykle wygrywa

Ten wariant jest znacznie bardziej odporny na tło niż czysty OpenCV, bo nie patrzy wyłącznie na „ruchome piksele”, tylko na **geometrię dłoni**. Dodatkowo MediaPipe ma typowe dla rozwiązań on-device optymalizacje potokowe i potrafi działać w czasie rzeczywistym przy małej rozdzielczości wejścia. Jeśli zechcesz dodać obsługę `Open_Palm` i `Closed_Fist`, zrób to nie przez ręczną analizę landmarków, lecz przez `GestureRecognizer`, bo ma te klasy wbudowane. ¹⁷

CPU optimization tips dla tego wariantu

Największy zysk dają tutaj: `num_hands=1`, rozdzielczość 640×360 lub 640×480, przetwarzanie co drugiej klatki, ograniczenie ROI do górnej/środkowej części kadru oraz przejście z `IMAGE` na `VIDEO` lub `LIVE_STREAM`. W trybie `LIVE_STREAM` nowe klatki mogą być odrzucane, jeśli inferencja nie nadąża; to bywa lepsze UX niż rosnący lag. Jeżeli chcesz mieć płynniejszy daemon, rozdziel czytanie z kamery i inferencję dwoma wątkami, ale w samym callbacku nie rób rzeczy ciężkich. ¹⁸

SDK specjalizowany z dodatkowym hardwarem

RealSense D435 lub D455 z `pyrealsense2`

RealSense SDK 2.0 jest cross-platform, ma oficjalne `pyrealsense2`, a dokumentacja i repozytorium są aktywne. D435/D455 pracują jako stereo depth; D4 wykonuje obliczenia depth na kamerze, co jest

korzystne dla hosta. D435 ma szerokie FOV i jest dobre dla szybkich scen; D455 jest lepsze do dłuższego zasięgu. Na Windows i Linux instalacja `pyrealsense2` jest prosta przez pip; na macOS istnieją warianty, ale oficjalna dokumentacja zaznacza, że wsparcie dla pełnej funkcjonalności jest niekompletne, a viewer i IMU są ograniczone. ¹⁹

Instalacja

```
pip install pyrealsense2 opencv-python numpy pynput
# macOS: w praktyce częściej trzeba użyć pyrealsense2-macosx
```

Kod

```
import time
from collections import deque

import cv2
import numpy as np
import pyrealsense2 as rs
from pynput.keyboard import Controller, Key

keyboard = Controller()

COOLDOWN_S = 0.8
MIN_AREA = 1500
MIN_DEPTH_MM = 200
MAX_DEPTH_MM = 800

history = deque(maxlen=8)
last_trigger = 0.0

def press_space():
    keyboard.press(Key.space)
    keyboard.release(Key.space)

pipeline = rs.pipeline()
config = rs.config()
config.enable_stream(rs.stream.depth, 640, 480, rs.format.z16, 30)

profile = pipeline.start(config)

spatial = rs.spatial_filter()
temporal = rs.temporal_filter()

try:
    while True:
        frames = pipeline.wait_for_frames()
        depth_frame = frames.get_depth_frame()
        if not depth_frame:
            continue
```

```

depth_frame = spatial.process(depth_frame)
depth_frame = temporal.process(depth_frame)

depth = np.asanyarray(depth_frame.get_data())
h, w = depth.shape

x0, y0 = int(w * 0.15), int(h * 0.20)
x1, y1 = int(w * 0.85), int(h * 0.85)
roi = depth[y0:y1, x0:x1]

# maska dłoni w zadanym zakresie odległości
mask = np.where((roi > MIN_DEPTH_MM) & (roi < MAX_DEPTH_MM), 255,
0).astype(np.uint8)
kernel = np.ones((5, 5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
debug = cv2.cvtColor(cv2.convertScaleAbs(depth, alpha=0.03),
cv2.COLOR_GRAY2BGR)

if contours:
    c = max(contours, key=cv2.contourArea)
    area = cv2.contourArea(c)

    if area > MIN_AREA:
        M = cv2.moments(c)
        if M["m00"] > 0:
            cx = int(M["m10"] / M["m00"])
            cy = int(M["m01"] / M["m00"])
            history.append((time.monotonic(), cx, cy))
            cv2.drawContours(debug[y0:y1, x0:x1], [c], -1, (0, 255,
0), 2)

            cv2.circle(debug[y0:y1, x0:x1], (cx, cy), 5, (0, 0,
255), -1)

        now = time.monotonic()
        if len(history) >= 5 and (now - last_trigger) > COOLDOWN_S:
            t0, cx0, cy0 = history[0]
            t1, cx1, cy1 = history[-1]
            dx = cx1 - cx0
            dy = cy1 - cy0

            if abs(dx) > 0.25 * (x1 - x0) and abs(dx) > 2.0 * abs(dy):
                press_space()
                last_trigger = now
                history.clear()

cv2.rectangle(debug, (x0, y0), (x1, y1), (255, 255, 0), 2)

```

```

cv2.putText(debug, "q = quit", (10, 25),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (50, 220, 50), 2)
cv2.imshow("realsense-depth-gesture", debug)

if cv2.waitKey(1) & 0xFF == ord("q"):
    break

finally:
    pipeline.stop()
    cv2.destroyAllWindows()

```

Kiedy wybrać RealSense

Wybierz RealSense, jeśli prezentujesz w trudniejszym tle, chcesz ograniczyć wpływ ubrań i sceny za Tobą albo masz już kamery depth. To bywa „nudniejsze” niż Ultraleap, ale bardzo pragmatyczne: depth robi dobrą robotę przy prostym gestzie kierunkowym bez dużego kodu ML. ²⁰

Ultraleap Leap Motion Controller 2 z LeapC w Pythonie

Ultraleap oferuje LeapC oraz repozytorium `leapc-python-bindings`, w którym jest oficjalny przykład `tracking_event_example.py` używający `leap.Connection()`, `leap.Listener` i `TrackingMode.Desktop`. To bardzo dobra ścieżka, jeśli akceptujesz specjalizowany hardware. Trzeba jednak uważać na zgodność wersji runtime i bindings: oficjalne repo bindings odnosi się do linii Gemini/LeapC i samo zaznacza, że prekompilowane moduły Pythona nie są jeszcze szeroko dostępne dla wszystkich wersji interpretera; dla Leap Motion Controller 2 Ultraleap zaleca obecnie runtime Hyperion.

²¹

Instalacja w skrócie

```

# 1) zainstaluj runtime Ultraleap (dla Controller 2 preferowany Hyperion)
# 2) sklonuj / pobierz official leapc-python-bindings
pip install -r requirements.txt
pip install -e leapc-python-api
# jeśli trzeba: python -m build leapc-cffi i doinstalowanie z dist/

```

Kod minimalny oparty na oficjalnym idiomie `leap.Listener`

```

import time
from collections import defaultdict, deque

import leap
from pynput.keyboard import Controller, Key

keyboard = Controller()
history = defaultdict(lambda: deque(maxlen=8))
last_trigger = 0.0
COOLDOWN_S = 0.7

```

```

def press_space():
    keyboard.press(Key.space)
    keyboard.release(Key.space)

class SwipeListener(leap.Listener):
    def on_tracking_event(self, event):
        global last_trigger
        now = time.monotonic()

        for hand in event.hands:
            hid = hand.id
            x = float(hand.palm.position.x)
            y = float(hand.palm.position.y)
            z = float(hand.palm.position.z)

            history[hid].append((now, x, y, z))
            if len(history[hid]) < 5 or (now - last_trigger) < COOLDOWN_S:
                continue

            t0, x0, y0, z0 = history[hid][0]
            t1, x1, y1, z1 = history[hid][-1]
            dx, dy, dz = x1 - x0, y1 - y0, z1 - z0

            # progi w milimetrach przestrzeni Leap
            if abs(dx) > 120 and abs(dx) > 2 * abs(dy) and abs(dx) > 2 *
abs(dz):
                press_space()
                last_trigger = now
                history[hid].clear()

def main():
    listener = SwipeListener()
    connection = leap.Connection()
    connection.add_listener(listener)

    with connection.open():
        connection.set_tracking_mode(leap.TrackingMode.Desktop)
        while True:
            time.sleep(1)

if __name__ == "__main__":
    main()

```

To jest bardzo obiecująca ścieżka jakościowo, ale jeśli priorytetem jest **najmniejsze ryzyko wdrożeniowe w Pythonie**, RealSense i MediaPipe są dziś zwykle prostsze w utrzymaniu. Ultraleap wygrywa wtedy, gdy chcesz **najwyższej jakości tracking dłoni** i akceptujesz poświęcenie trochę czasu na środowisko. ²²

Strojenie wydajności

Największy zysk wydajnościowy w praktyce nie bierze się z „lepszego GPU”, tylko z **ograniczenia ilości danych**, które przechodzą przez pipeline. To oznacza: mała rozdzielczość wejścia, małe ROI, pojedyncza ręka, mała liczba inferencji na sekundę i prosty automat stanów zamiast zaawansowanego klasyfikatora sekwencji. OpenCV samo w sobie jest biblioteką cross-platform, a MediaPipe Tasks są przygotowane do on-device execution na CPU/GPU/TPU. W Twoim use case najlepsza strategia to zwykle: **CPU-first, model-light, ROI-first**. ²³

W MediaPipe używaj `VIDEO` albo `LIVE_STREAM`, nie `IMAGE`. Oficjalna dokumentacja wprost mówi, że w `VIDEO` i `LIVE_STREAM` wykorzystywany jest tracking, który ogranicza częstotliwość uruchamiania modelu palm detection. W `LIVE_STREAM` `detect_async()` zwraca natychmiast, a task może odrzucać część klatek, jeśli to obniża latencję. To bardzo sensowne zachowanie dla „background daemonu”, gdzie ważniejsza jest responsywność niż 100% pokrycia wszystkich klatek. ²⁴

Jeżeli z jakiegoś powodu wyjdiesz poza OpenCV/MediaPipe i zrobisz własny model, to optymalizuj go pod inferencję, a nie pod „model accuracy” w próżni. TensorFlow Lite zaleca post-training quantization przez `tf.lite.Optimize.DEFAULT`, a full integer quantization może poprawić latencję, zużycie energii i wykorzystanie akceleratorów integer-only. ONNX Runtime ma analogiczne API do kwantyzacji modeli FP32 do INT8 i własne optymalizacje grafu. W praktyce oznacza to: **zanim sięgniesz po większy model, spróbuj kwantyzacji i prostszego runtime’u**. ²⁵

W wersji systemowej polecam kilka twardych zasad: odczyt z kamery w osobnym wątku, inferencja lub analiza ruchu w drugim; limit fps logicznego do 10–15 dla prezentacji; rozdzielczość 640×360 lub 640×480; wysyłanie klawisza tylko po 1–2 potwierdzeniach stanu; i bez żadnych ciężkich operacji w callbackach. `pynput` wyraźnie ostrzega, że długie procedury w callbackach mogą na niektórych platformach zamrozić wejście dla procesów. ²⁶

Jeżeli jednak dokupujesz hardware, to właśnie dlatego **RealSense** i **Ultraleap** są tak ciekawe: duża część ciężaru percepcji jest przeniesiona poza klasyczny „pełny obraz RGB z laptopa”. RealSense ma depth obliczane na kamerze przez D4, a Ultraleap ma dedykowany stos do śledzenia dłoni i bardzo niskie wymagania hosta. To często jest bardziej efektywne niż próba „ratowania” zwykłej kamery coraz cięższym modelem. ²⁷

Bezpieczeństwo, prywatność i UX

W produkcie tego typu najważniejszy jest nie „FPS”, tylko **brak fałszywych aktywacji**. Dlatego obowiązkowo wprowadź **debounce** i **cooldown**. Sensowna wartość na start to 700–1000 ms między kolejnymi aktywacjami. Dodatkowo bardzo pomaga warunek „gesture confirmation”, np. otwarta dłoń przez krótki czas, a dopiero potem swipe. W MediaPipe jest to wygodne, bo Gesture Recognizer ma gotowe klasy `Open_Palm` i `Closed_Fist`. ²⁸

Od strony prywatności modalności różnią się wyraźnie. **RGB webcam** widzi pełną scenę i twarz. **Ultraleap** operuje na stereo IR. **RealSense** może pracować na depth, a nie na pełnym kolorze. **Soli** ma tu największą przewagę koncepcyjną: Google wprost podkreśla, że Motion Sense nie jest kamerą i nie generuje rozróżnialnych obrazów ciała lub twarzy do detekcji obecności i gestów. Jeśli prezentujesz w środowisku z silnymi wymaganiami prywatności, radar lub IMU są lepsze niż RGB, ale kosztują Cię dostępnością i prostotą wdrożenia. ²⁹

Od strony systemowej musisz uwzględnić uprawnienia. Na macOS aplikacja sterująca klawiaturą zwykle potrzebuje zgody w **Privacy & Security → Accessibility**. Na Windows wstrzykiwanie wejścia przez `SendInput` podlega **UIPI**, więc aplikacja może wysyłać input tylko do procesów na równym lub niższym poziomie integralności. Na Linuksie `pynput` używa backendów `xorg` lub opcjonalnego `uinput`, przy czym `uinput` wymaga roota i obsługuje tylko klawiatury. To wszystko warto sprawdzić przed demo na cudzym laptopie. ³⁰

W UX polecam jeszcze cztery drobne reguły. Po pierwsze, **dawaj feedback** — krótki dźwięk, overlay w rogu albo ikonę tray. Po drugie, **uzależnij aktywność od fokusowanego okna**, żeby przypadkowy gest nie przełączył czegoś poza prezentacją. Po trzecie, **wyłączaj daemon podczas nagrywania ekranu lub screen share**, jeśli nie jest potrzebny. Po czwarte, daj użytkownikowi **globalny skrót aktywacja/dezaktywacja**. To są rekomendacje projektowe, ale bardzo często ważniejsze od samego modelu. ³¹

Pipeline i harmonogram MVP

Poniższy pipeline jest dobrym minimalnym wzorcem dla wszystkich wariantów — od OpenCV po MediaPipe i depth.

```

flowchart TD
    A[Kamera lub sensor] --> B[Przechwycenie klatki lub próbki]
    B --> C[Downscale i ROI]
    C --> D{Modalność}
    D -->|RGB heurystyczne| E[Foreground mask lub optical flow]
    D -->|RGB + MediaPipe| F[Landmarki dłoni lub gesty statyczne]
    D -->|Depth lub IR| G[Segmentacja po głębi lub położeniu dłoni]
    E --> H[Ekstrakcja cech czasowych]
    F --> H
    G --> H
    H --> I[Automat stanów gestu]
    I --> J[Debounce i cooldown]
    J --> K[Sprawdzenie trybu aktywnego]
    K --> L[Wysłanie Space]
    L --> M[Feedback i log zdarzenia]

```

Minimalny harmonogram dla działającego prototypu na laptopie bez dGPU wygląda realistycznie tak:

zadanie	szacowany czas
bootstrap repo, kamera, wysyłanie <code>Space</code> , logi	2–3 h
prototyp OpenCV z ROI i debounce	3–5 h
prototyp MediaPipe z landmarkami i swipe	4–8 h
strojenie progów, false positive, cooldown	3–5 h
tray toggle / hotkey / focus gating	2–4 h
smoke test Windows + macOS lub Windows + Linux	3–6 h
packaging do jednego procesu uruchamianego w tle	2–4 h

Daje to **około 19–35 godzin** do sensownego MVP, jeśli zaczynasz od zera, oraz **8–14 godzin**, jeśli budujesz wyłącznie wersję MediaPipe na jednym systemie operacyjnym i akceptujesz tryb „debug window”. To jest estymata inżynierska, nie benchmark. Największe ryzyka harmonogramu to zwykle nie CV, tylko uprawnienia systemowe do synthetic input, testy na różnych kamerach i walka z false positive.

32

Priorytetowe źródła

Najbardziej wartościowe źródła do tego tematu to przede wszystkim dokumentacja producentów i publikacje pierwotne. Dla **MediaPipe**: przewodniki Hand Landmarker i Gesture Recognizer dla Pythona, benchmark Gesture Recognizer oraz paper o MediaPipe Hands. 33

Dla **OpenCV**: oficjalne tutoriale o VideoCapture, background subtraction i optical flow oraz dokumentacja Video I/O backendów. 34

Dla **Ultraleap**: Getting Started, Hyperion overview, TouchFree User Manual, LeapC concepts oraz oficjalne leapc-python-bindings z przykładami tracking_event_example.py. 35

Dla **RealSense**: Supported operating systems, D435/D455/D435i product pages, repo librealsense, pyrealsense2 na PyPI i python wrapper README. 36

Dla ścieżek **badawczo-specjalistycznych**: Google Soli / Motion Sense, lekki radar FMCW on-edge, HUG dla ultradźwięków i rękawica IMU. 37

Jeżeli chcesz źródła **po polsku**, najpraktyczniejsze są dziś raczej sklepy i dystrybutorzy niż oficjalne dokumentacje. Użyteczne jako punkt odniesienia dla lokalnej dostępności są m.in. karta **Leap Motion Controller 2 w Botlandzie** oraz polskie listingi modułów **IMU BNO085** w Kamami/Botland. Trzeba jednak traktować je pomocniczo wobec źródeł producenta. 38

1 8 <https://pypi.org/project/mediapipe/>
<https://pypi.org/project/mediapipe/>

2 10 22 35 <https://docs.ultraleap.com/hand-tracking/getting-started.html>
<https://docs.ultraleap.com/hand-tracking/getting-started.html>

3 37 <https://research.google/blog/soli-radar-based-perception-and-interaction-in-pixel-4/>
<https://research.google/blog/soli-radar-based-perception-and-interaction-in-pixel-4/>

4 15 16 18 24 33 https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/python
https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/python

5 <https://arxiv.org/pdf/2310.08876>
<https://arxiv.org/pdf/2310.08876>

6 28 https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer
https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer

7 13 14 https://docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html
https://docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html

9 23 <https://opencv.org/platforms/>
<https://opencv.org/platforms/>

- 11 26 31 <https://pypi.org/project/pynput/>
<https://pypi.org/project/pynput/>
- 12 https://docs.opencv.org/4.x/d1/dc5/tutorial_background_subtraction.html
https://docs.opencv.org/4.x/d1/dc5/tutorial_background_subtraction.html
- 17 <https://research.google/pubs/mediapipe-hands-on-device-real-time-hand-tracking/>
<https://research.google/pubs/mediapipe-hands-on-device-real-time-hand-tracking/>
- 19 <https://github.com/realsenseai/librealsense/blob/master/wrappers/python/readme.md>
<https://github.com/realsenseai/librealsense/blob/master/wrappers/python/readme.md>
- 20 <https://www.realsenseai.com/products/stereo-depth-camera-d435/>
<https://www.realsenseai.com/products/stereo-depth-camera-d435/>
- 21 https://github.com/ultraleap/leapc-python-bindings/blob/main/examples/tracking_event_example.py
https://github.com/ultraleap/leapc-python-bindings/blob/main/examples/tracking_event_example.py
- 25 https://www.tensorflow.org/model_optimization/guide/quantization/post_training
https://www.tensorflow.org/model_optimization/guide/quantization/post_training
- 27 <https://www.realsenseai.com/products/real-sense-depth-camera-d455f/>
<https://www.realsenseai.com/products/real-sense-depth-camera-d455f/>
- 29 <https://support.google.com/googlenest/answer/10388741?hl=en>
<https://support.google.com/googlenest/answer/10388741?hl=en>
- 30 32 <https://support.apple.com/guide/mac-help/allow-accessibility-apps-to-access-your-mac-mh43185/mac>
<https://support.apple.com/guide/mac-help/allow-accessibility-apps-to-access-your-mac-mh43185/mac>
- 34 https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html
https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html
- 36 <https://dev.realsenseai.com/installation/supported-operating-systems/>
<https://dev.realsenseai.com/installation/supported-operating-systems/>
- 38 <https://botland.com.pl/produkty-wycofane/23818-leap-motion-controller-2-precyzyjny-czujnik-gestow-ultraleap-5060925710021.html>
<https://botland.com.pl/produkty-wycofane/23818-leap-motion-controller-2-precyzyjny-czujnik-gestow-ultraleap-5060925710021.html>